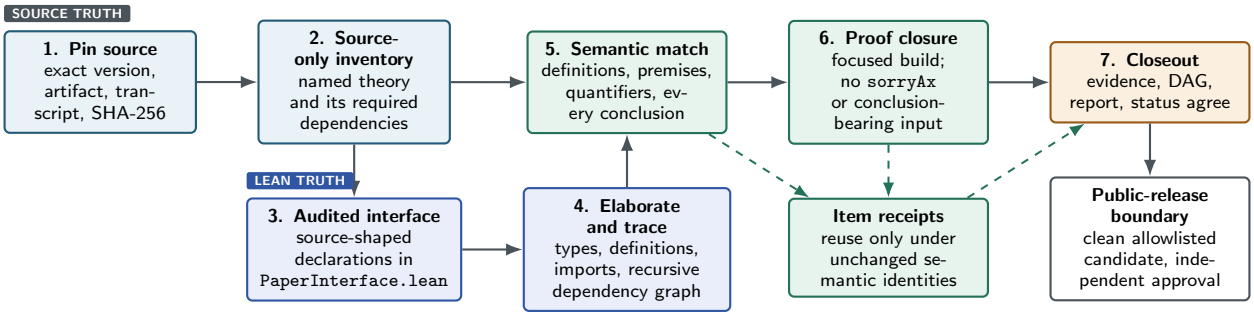
# How a Paper Formalization Is Audited

High-level procedure, August 2026

**Purpose.** The audit answers three different questions: (i) did we select the right claims from the paper, (ii) do the elaborated Lean declarations say the same thing, and (iii) has Lean proved those declarations without hidden mathematical inputs? A successful build answers only the third question for the statements that were actually encoded. Source fidelity and release certification require their own evidence.




Solid arrows are required gates. Dashed arrows carry reusable evidence; they never bypass a gate.

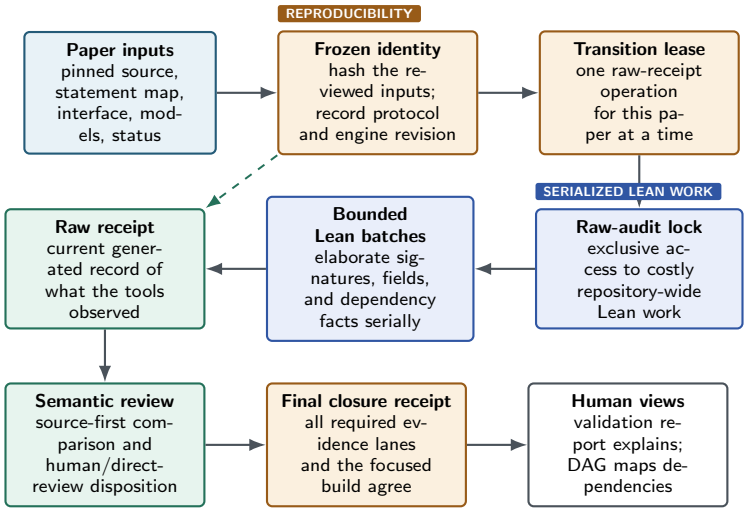
**The stages.**

1. **Pin before interpreting.** Record the exact paper version and a byte-addressed, reviewable source artifact. A PDF may establish provenance, but normal inventory uses stable text or TEX whose derivation is recorded.
2. **Inventory from the source, not from the code.** Independently enumerate the normal surface: source-presented named definitions, models, assumptions, lemmas, propositions, theorems, corollaries, and theorem-like claims. Include an unlabelled item when a selected result depends on it. Figures, simulations, numerical examples, empirical claims, and standalone displays or algorithms are separate obligations only in explicit deep mode.
3. **Write the review surface first.** Put the actual paper-facing definitions and theorem types, in paper order, in `PaperInterface.lean`. Proof plumbing belongs elsewhere. A private `by sorry` body may temporarily freeze an audited statement; it is a skeleton, never completion evidence.
4. **Audit elaborated meaning.** Lean elaborates the declarations, and the manifest records their name-independent proposition structure, reducible definitions, repository-local transitive dependencies, opaque imported terminals, and toolchain context. Recursive inspection follows record fields and aliases rather than trusting wrapper types.
5. **Join source and Lean item by item.** Coverage passes only when one complete pinned source presentation, one current elaborated declaration, and the premise/conclusion ledgers agree semantically. Names, numbering, routes, and similar prose are navigation aids, not evidence.
6. **Close the mathematics.** Replace every temporary placeholder, build the complete paper target, and verify that the dependency closure has no `sorryAx`, axiom-like escape, or input record whose fields assume the advertised conclusion. A proof-body change reruns proof closure but need not invalidate an unchanged statement judgment.
7. **Close evidence before release.** Run the consolidated paper closeout, an independent source-first holistic audit, and the paper DAG/report checks. The canonical evidence, final report, and `status.json` must give one consistent account of assumptions, clarifications, boundaries, review, and mathematical status.

## How the Audit Operation Runs

Operational mechanics, not another mathematical gate

The first diagram is the *logical* audit flow: what must be established about a paper. This diagram is the *operational* flow: how the repository creates one reproducible, current evidence record without two tools racing to write it. These mechanics protect the audit; they do not themselves establish that a source-to-Lean bridge is mathematically correct.




Solid arrows are operations that produce or check evidence. The dashed arrow records the exact frozen context that makes a raw receipt reusable.

**What the operational layer does—and does not do.** The audit engine checks a frozen, named review surface according to the review protocol. The lease, lock, and batches ensure that two executions cannot silently produce incompatible records or exhaust the shared Lean environment. They do not choose the paper's claims, decide the intended mathematics, or substitute for the source-first semantic review.

**Receipt hierarchy.** The raw receipt is the detailed tool observation for one frozen input identity. The final closure receipt names that raw evidence and the required review lane, then records the paper-level disposition. The validation report is a readable explanation of that decision; the DAG is a readable map of the claims and their dependencies.

## Operational Glossary

The vocabulary used by the implementation

| Term | Meaning in this workflow |
|---|---|
| **Audit engine** | The versioned implementation of the audit checks. Its code hash and revision record make a run reproducible. It is an operational guard, not mathematical evidence and not a verdict on the paper. |
| **Review protocol** | The machine-readable policy defining audit scope and what a current closeout must check. A protocol change can change obligations; an engine change can instead be implementation-compatible. Both are recorded explicitly. |
| **Freeze / input identity** | A snapshot of the precise source artifact, map, review interface, source-model files, relevant configuration, and imported Lean closure. If one of these changes, a prior raw receipt is stale for the new inputs rather than silently reused. |
| **Transition lease and lock** | A lease prevents two closeout wrappers from trying to preserve/reissue the same paper at once. The raw-audit lock separately serializes expensive Lean scans across the repository. Their heartbeats are progress diagnostics, not evidence. |
| **Lean batch** | One bounded worker request, normally for a small exact set of declarations. Batching controls memory and makes interruptions recoverable; it does not split or weaken a mathematical review. |
| **Raw receipt** | The generated, machine-readable source-record audit bound to the frozen inputs. It reports elaboration and routing facts, including failures. It is necessary evidence, but it is not the final human-facing validation. |
| **Final closure receipt** | The one canonical machine-readable closeout record. It binds the current raw evidence, required semantic/direct-review lane, focused Lean build, and status disposition. It replaces competing notions of a "final report" or an informal receipt. |

**Why a fresh receipt is sometimes required.** Changing a proof body can leave a statement-level review current, while changing the source, map, interface, source model, selected audit policy, or relevant imported Lean content changes the thing being audited. The workflow then records a new frozen identity and reissues the raw receipt. A registered engine transition is also placed in a new operational wave so that the record says exactly which audited implementation produced it. This bookkeeping does *not* erase prior mathematical work or imply that the paper became false; it prevents an old machine observation from being presented as if it described changed inputs.

**What to read.** For a quick understanding of a paper, read the validation report and DAG. For the exact closeout decision, read the final closure receipt it names. The raw receipt and operation trace answer narrower questions such as "what did the tool see?" and "is a reissue still running?"; they are useful diagnostics, not the primary narrative for a reader.

## What Must Match

Semantic content, not identifiers

| Dimension | Required agreement | Typical failure caught |
|---|---|---|
| **Source identity** | Exact version, full byte-pinned presentation, source-only scope classification, and stable locator. | Convenient excerpt omits a hypothesis; map was built by looking at Lean names. |
| **Definitions and model** | Objects, domains, encodings, normalization, tie-breaking, state evolution, and relevant source conventions have the same mathematical meaning. | A wrapper silently strengthens the model; a representation changes the admissible inputs. |
| **Premises** | Every source premise is represented; every visible or recursively packed Lean premise is source-backed or proved from visible source premises by an explicit checked bridge. | "Certificate," record, or helper theorem contains the desired conclusion as an assumption. |
| **Conclusions** | Quantifiers, inequalities/equalities, cases, output guarantees, and *every* source conclusion atom are recovered. | Lean proves only one direction, a restricted case, or existence without the advertised output. |
| **Algorithms and cost** | When part of a selected named result, operational semantics, termination, numeric model, and cost scope match as well as the final mathematical relation. | A noncomputable existence wrapper is credited as a polynomial-time algorithm. |

### Reuse Without Re-auditing Everything

Review receipts are cached *per source item*. Reuse is allowed only when all material identities remain unchanged:

- source semantics and the byte-validated quote;
- elaborated, name-independent statement meaning;
- transitive semantic dependency graph and imported terminal artifacts;
- model/correction dispositions, protocol, validator, and toolchain context; and
- a unique current route to the same item.

A declaration, file, or map-key rename may be rebound only through a unique semantic match. Missing, changed, or ambiguous identity fails closed. Thus a small change reopens that item and affected dependents, not every unchanged row. Reuse saves work; it is not independent evidence of correctness.

### Distinct Verdicts

- **Lean-closed:** all intended endpoints are proved under the displayed formal assumptions.
- **Source-audited:** source inventory, statements, premises, conclusions, definitions, and dependencies have current semantic evidence.
- **Human-reviewed:** a human has compared the pinned source with the complete review surface. LLM judgments are useful audit evidence, not a substitute for that review.
- **Release-certified:** source bytes are verified, required roles and approvals are recorded, and the exact public candidate passes the release boundary.

These axes are reported separately. A paper may be Lean-closed while source audit or release certification is still pending. Conversely, fresh audit sidecars cannot make an open proof *Formalized*.

**Public-release boundary.** Completed work is selected from the private incubator into a clean branch based on public `main`. A reviewed path allowlist, current canonical evidence, source-byte/license hygiene, the release guard, focused and integration builds, status synchronization, and explicit human approval bind the exact candidate. Private source artifacts, scratch work, historical audit snapshots, and unrelated papers are not carried into the public commit.

Normative operational details live in `config/formalization_audit_protocol.json`. The principal human guides are `VALIDATION_MODEL.md`, `PAPER_COVERAGE_AUDIT.md`, `INDEPENDENT_AUDIT_GUIDE.md`, and `PUBLIC_RELEASE_CHECKLIST.md`.